Comparaison de trois algorithmes de calcul de l'enveloppe convexe

Jolis Algorithmes Classiques

Caio Medeiros Balaniuk

1. Introduction et objectif

L'objectif de ce projet est d'implémenter et de comparer trois algorithmes classiques qui permettent de calculer l'enveloppe convexe d'un ensemble de points : Jarvis March, Andrew's Monotone Chain et Graham Scan.

L'enveloppe convexe est le plus petit polygone convexe qui contient tous les points d'un ensemble. On peut l'imaginer comme un élastique placé autour de plusieurs points : lorsqu'on le relâche, il prend la forme de l'enveloppe convexe.

Ce problème est utilisé dans plusieurs domaines, par exemple la géométrie, la robotique, la reconnaissance d'images et la modélisation informatique.

Le but du projet est de comprendre le fonctionnement de chaque algorithme, de comparer leur difficulté d'implémentation et d'étudier leurs performances avec un grand nombre de points.

2. Présentation des algorithmes

2.1. Jarvis March

Jarvis March, aussi appelé *Gift Wrapping*, construit l'enveloppe convexe en sélectionnant les sommets les uns après les autres.

L'algorithme commence par un point extrême, généralement celui qui possède la plus petite coordonnée en x. Ensuite, il cherche le prochain sommet en comparant les directions entre les points. Ce processus continue jusqu'au retour au point de départ.

Sa complexité temporelle est (O(nh)), où (n) représente le nombre total de points et (h) le nombre de sommets de l'enveloppe convexe.

Cet algorithme est assez facile à comprendre, mais ses performances dépendent du nombre de sommets de l'enveloppe.

2.2. Andrew’s Monotone Chain

L’algorithme d’Andrew construit l’enveloppe convexe en deux parties : la partie inférieure et la partie supérieure.

Tout d’abord, les points sont triés selon leurs coordonnées x, puis selon leurs coordonnées y si nécessaire. Ensuite, l’algorithme construit les deux parties de l’enveloppe en vérifiant l’orientation des points. Si un point ne respecte pas la forme convexe, il est retiré de la construction.

Sa complexité temporelle est (O(n \log n)), principalement à cause du tri initial.

Cet algorithme est relativement simple à implémenter et offre de bonnes performances, même pour un grand nombre de points.

2.3. Graham Scan

Graham Scan commence par choisir un point de référence, généralement celui qui possède la plus petite coordonnée y. Les autres points sont ensuite triés selon leur angle polaire par rapport à ce point.

L’algorithme parcourt les points triés et utilise une pile pour construire l’enveloppe convexe. Lorsqu’un point crée un mauvais changement de direction, le dernier point ajouté à la pile est retiré.

Sa complexité temporelle est (O(n \log n)), car le tri des points domine généralement le calcul.

Graham Scan est efficace, mais son implémentation demande plus d’attention, notamment pour le calcul des angles, le tri et la gestion des changements de direction.

3. Méthode expérimentale

Les trois algorithmes ont été implémentés en langage C. Pour comparer leurs performances, un ensemble de 50 000 points a été généré aléatoirement.

Les coordonnées des points sont des nombres entiers générés dans un intervalle allant de 0 à 999 999. La même méthode de génération est utilisée pour obtenir des résultats comparables.

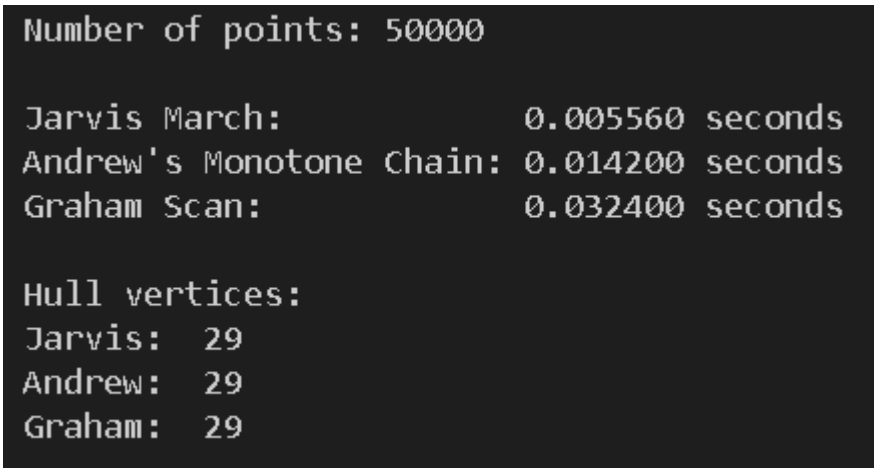
Chaque algorithme est exécuté 100 fois. Le temps moyen d’exécution est ensuite calculé à partir des mesures réalisées avec la fonction `clock()` du langage C.

Après les calculs, les résultats des trois algorithmes sont également comparés pour vérifier qu’ils trouvent la même enveloppe convexe. Dans notre expérience, les trois algorithmes trouvent 29 sommets et produisent des enveloppes équivalentes.

Enfin, un programme en Python utilisant la bibliothèque Matplotlib permet de visualiser les points générés et l'enveloppe convexe obtenue.

4. Comparaison des performances

Le photo suivant présente les temps moyens obtenus lors d'une des exécutions du programme.



Conditions du test : 50 000 points, 100 répétitions et 29 sommets dans l'enveloppe convexe.

Dans cette expérience, Jarvis March est le plus rapide, suivi par Andrew's Monotone Chain et Graham Scan.

Ce résultat peut sembler surprenant, car Jarvis March possède une complexité (O(nh)). Cependant, le nombre de sommets de l'enveloppe est relativement petit par rapport au nombre total de points. L'algorithme doit donc effectuer relativement peu d'étapes de recherche de sommets.

Andrew's Monotone Chain présente également de bonnes performances. Graham Scan est plus lent dans cette implémentation, notamment à cause du tri par angle et des opérations supplémentaires nécessaires à la construction de l'enveloppe.

Il est important de noter que ces temps dépendent de l'ordinateur, du compilateur, de l'implémentation et de la distribution des points. Ils ne permettent donc pas de conclure que Jarvis March est toujours le plus rapide.

5. Difficulté de programmation

Les trois algorithmes présentent des difficultés différentes lors de leur implémentation en C.

Jarvis March est le plus simple à comprendre. Son principe consiste à rechercher le prochain sommet de l'enveloppe en comparant les orientations. La principale difficulté est de gérer correctement les cas particuliers, comme les points alignés.

Andrew's Monotone Chain est également assez simple à programmer. Le tri initial est facile à réaliser et la construction des parties supérieure et inférieure suit une logique claire. Il faut cependant bien gérer les orientations et les points alignés.

Graham Scan est plus délicat à implémenter. Il faut choisir un point de référence, trier les autres points selon leur angle polaire et utiliser une pile pour construire l'enveloppe. Le tri et la gestion des cas particuliers demandent plus d'attention.

En résumé, Jarvis March est le plus intuitif, Andrew's Monotone Chain offre un bon équilibre entre simplicité et efficacité, et Graham Scan demande davantage de précautions dans son implémentation.

6. Visualisation des résultats

Le programme Python utilise Matplotlib pour afficher les points générés et l'enveloppe convexe calculée.

Les points de l'ensemble sont représentés en bleu, tandis que l'enveloppe convexe est représentée par une ligne rouge. Les sommets de l'enveloppe sont également mis en évidence.

Cette visualisation permet de vérifier la forme de l'enveloppe et de mieux comprendre le résultat produit par les algorithmes.

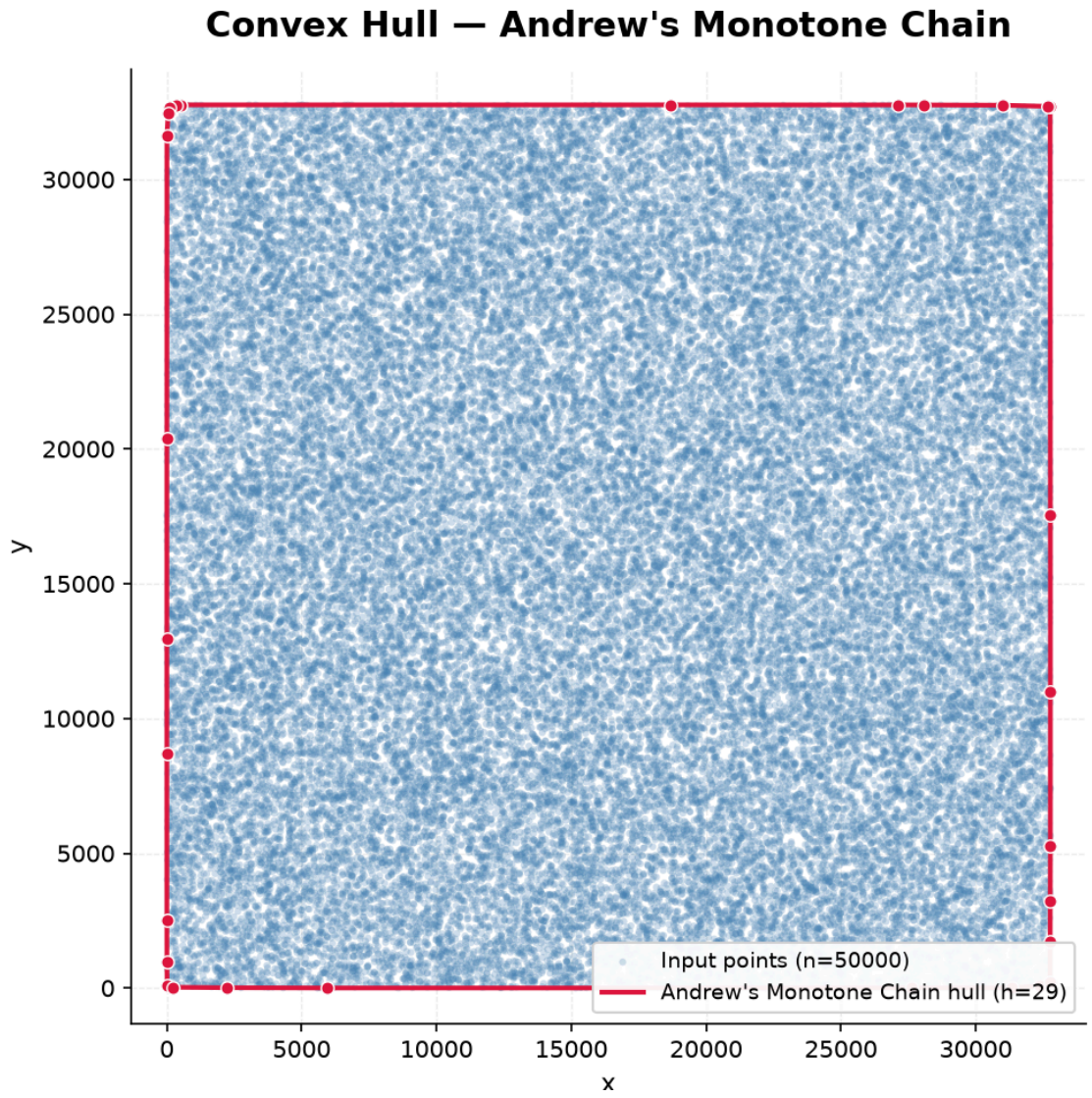


Figure 1 — Visualisation des 50 000 points et de leur enveloppe convexe (Andrews Chain)

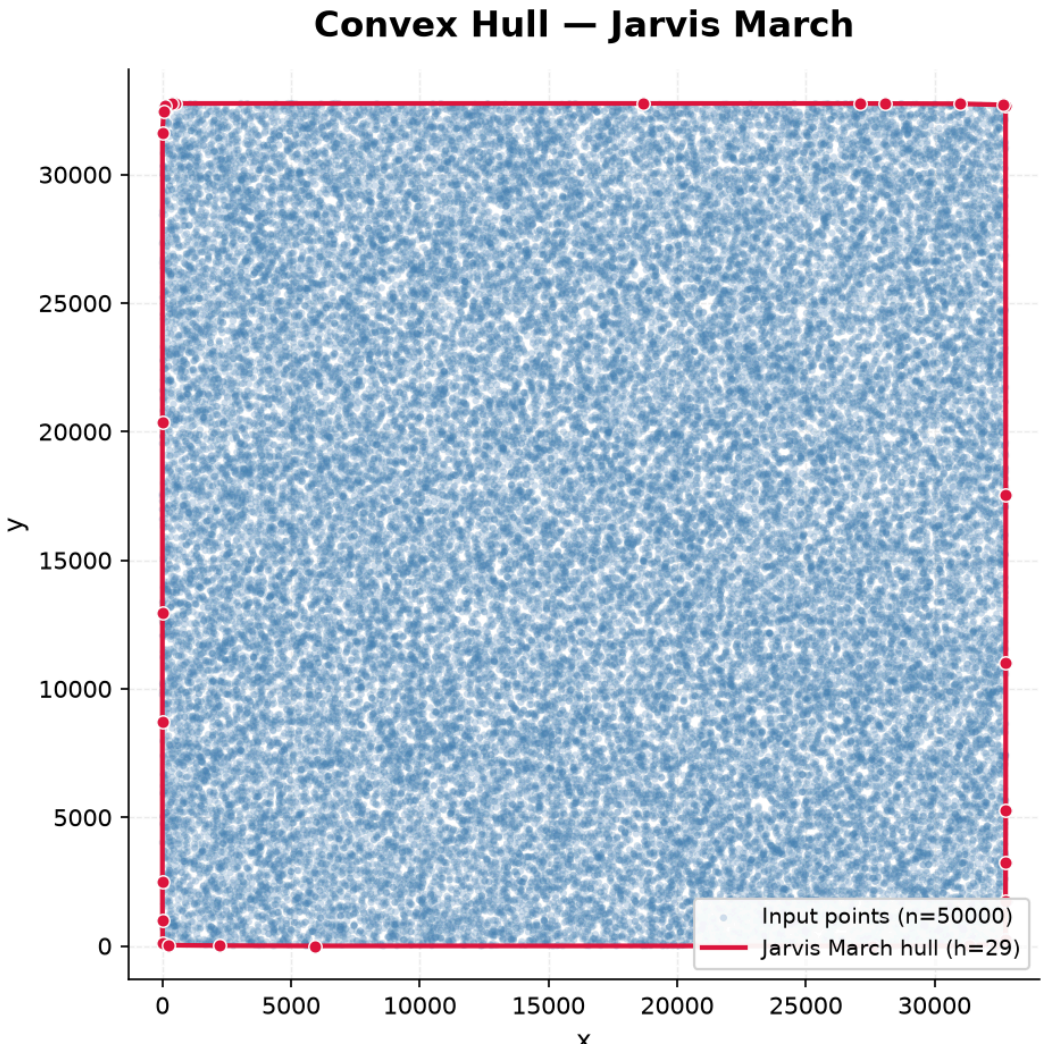


Figure 2 — Visualisation des 50 000 points et de leur enveloppe convexe (Jarvis March)

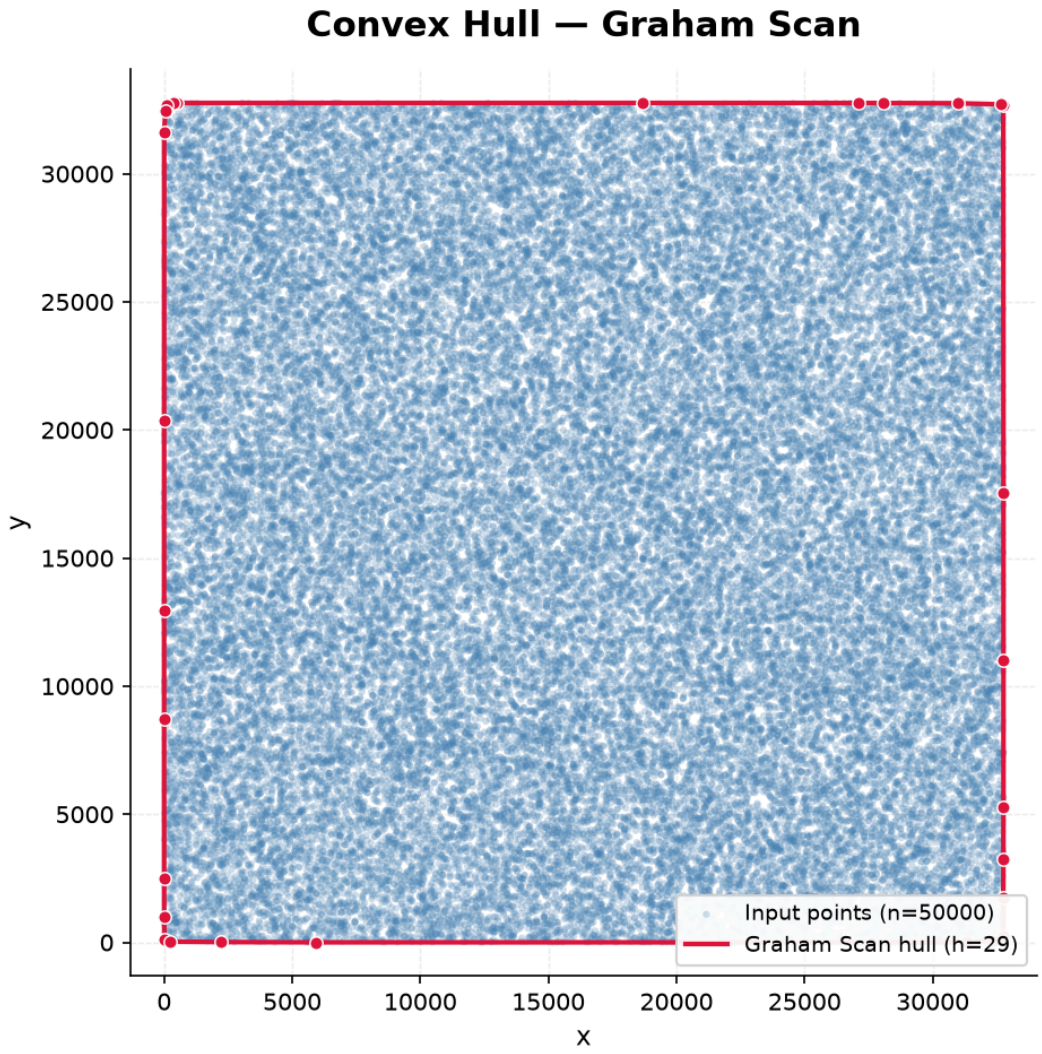


Figure 3 — Visualisation des 50 000 points et de leur enveloppe convexe (Graham Scan)

On peut observer que les graphiques obtenus avec les trois algorithmes sont pratiquement identiques. Cela est logique, car les trois algorithmes ont pour objectif de calculer l'enveloppe convexe du même ensemble de points. Même si leurs méthodes de calcul sont différentes, ils doivent obtenir le même résultat. Dans notre expérience, les trois algorithmes trouvent les mêmes 29 sommets, ce qui confirme que leurs résultats sont équivalents pour cet ensemble de données.

7. Conclusion

Ce projet nous a permis d'implémenter et de comparer trois algorithmes classiques de calcul de l'enveloppe convexe.

Les expériences montrent que les performances ne dépendent pas uniquement de la complexité théorique. Dans notre test, Jarvis March est le plus rapide, car l'enveloppe convexe contient seulement 29 sommets pour 50 000 points. Andrew's Monotone Chain offre également de bonnes performances, tandis que Graham Scan est le plus lent dans notre implémentation.

Du point de vue de la programmation, Jarvis March est le plus intuitif, Andrew's Monotone Chain est relativement simple et efficace, et Graham Scan demande une gestion plus complexe du tri et de la pile.

Cependant, cette expérience présente certaines limites. Les résultats sont obtenus avec une seule distribution de points et dépendent de l'ordinateur et de l'implémentation. Pour approfondir la comparaison, il serait intéressant de tester plusieurs tailles d'ensembles et différentes distributions de points, notamment des ensembles dont l'enveloppe convexe contient beaucoup plus de sommets.

En conclusion, ce projet montre l'importance de comparer les algorithmes à la fois sur le plan théorique et expérimental. Il permet également de mieux comprendre l'influence de la structure des données sur les performances des algorithmes géométriques.